Industr.IA

E-book

IA Generativa:

prompting, RAG e fine-tuning como padrões aplicados



Curso base de IA | Unidade M1

Prof.: Arthur Nascimento

SUMÁRIO

1. Prompting estruturado 3
2. RAG Mínimo 6
3. Fine-tuning leve 9
4. Reconhecimento de padrões em uma especificação 11
4.1 Sinais na especificação e o padrão que cada um indica. 12
Conclusão 14

Unidade M1

1. Prompting estruturado

Falaremos um pouco sobre camada aplicada. O principal objetivo deste tópico é que você reconheça o prompting estruturado como o primeiro dos três padrões que organizam soluções generativas em PD&I industrial, identificando seus componentes mínimos e os sinais de que ele já não basta.

Prompting estruturado é o padrão de menor custo de montagem entre os três, e por isso o ponto de partida natural de quase todo projeto generativo. A ideia é obter o comportamento desejado apenas pela composição da entrada que se apresenta ao modelo, sem alterar seus pesos e sem acrescentar conhecimento externo em tempo de execução. Tudo o que o modelo sabe vem do seu conhecimento paramétrico, fixado no pré-treino; tudo o que ele faz com a tarefa vem da forma como o pedido foi escrito. O adjetivo estruturado distingue esse padrão do uso improvisado de um chatbot: em vez de um texto livre, o prompt é montado com partes nomeadas e estáveis, que podem ser versionadas, testadas e auditadas como qualquer outro artefato de software.

Quatro componentes aparecem na maioria dos prompts industriais. A instrução de sistema define o papel, as restrições e a política de resposta, valendo para toda a sessão. A instrução de tarefa descreve o que fazer com a entrada específica. Os exemplos, quando presentes, mostram pares de entrada e saída no formato esperado; é o in-context learning já visto no módulo M0, em regime few-shot quando há vários exemplos e zero-shot quando não há nenhum. O quarto componente, muitas vezes esquecido, é o contrato de saída: a especificação explícita do formato que a resposta deve ter para ser consumida pelo restante do sistema.

GLOSSÁRIO

Saída estruturada

Saída estruturada (structured output) é a resposta que o modelo produz em formato verificável por máquina, como um objeto JSON com campos obrigatórios e valores restritos a um conjunto fechado, em vez de texto em prosa. O function calling é o mecanismo que impõe esse contrato.

O contrato de saída é o que separa uma demonstração de uma solução. Em PD&I, a resposta do modelo raramente é o produto final: ela alimenta um pipeline a jusante que indexa, agrega ou aciona outro sistema. Texto livre quebra esse pipeline a cada variação

de redação; uma saída estruturada, validada contra um schema, transforma o modelo em um componente previsível. Quando o formato é crítico, pedir o formato no prompt e ainda validar a saída fora do modelo é a prática mínima, porque o modelo respeita o contrato na maioria das vezes, não em todas.

Tratar o prompt como software tem consequências concretas. Cada parte nomeada passa a ser um artefato sob controle de versão, com histórico de mudanças e responsável definido. Antes de promover uma alteração, compara-se a saída nova contra um conjunto de casos de referência, do mesmo modo que se roda uma suíte de testes de regressão; sem essa disciplina, uma melhora local pode degradar casos que já funcionavam sem que ninguém perceba até a produção. A reprodutibilidade depende também dos parâmetros de geração: uma temperatura alta aumenta a variabilidade das respostas, útil para exploração e nociva para tarefas que exigem saída estável, nas quais se reduz a temperatura e se fixa a versão do modelo.

Quadro 1 - Anatomia de um prompt estruturado.

Componente	Função	Exemplo de Conteúdo
Instrução de sistema	Fixa papel, restrições e política para toda a sessão	Você é um triador de chamados; nunca invente campos ausentes
Instrução de tarefa	Descreve o que fazer com a entrada atual	Classifique a severidade e extraia o equipamento citado
Exemplos (few-shot)	Calibram formato e fronteira de decisão	Dois chamados resolvidos, com a saída no formato alvo
Contrato de saída	Especifica o formato verificável da resposta	JSON com campos severidade (enum) e equipamento (string)

Para Cego Ver: A imagem apresenta uma tabela intitulada "Quadro 1. Anatomia de um prompt estruturado". A tabela possui três colunas: Componente, Função e Exemplo de conteúdo. São apresentados quatro componentes de um prompt estruturado. O primeiro é a Instrução de sistema, responsável por definir o papel da inteligência artificial, as restrições e as políticas que permanecem válidas durante toda a interação, ilustrado pelo exemplo "Você é um triador de chamados; nunca invente campos ausentes". O segundo é a Instrução de tarefa, que descreve o que deve ser feito com a entrada fornecida, exemplificada por "Classifique a severidade e extraia o equipamento citado". O terceiro componente são os Exemplos (few-shot), utilizados para orientar o formato esperado da resposta e os critérios de decisão, com o exemplo de dois chamados resolvidos apresentados no formato desejado. Por fim, o Contrato de saída especifica como a resposta deve ser estruturada, utilizando como exemplo um objeto JSON contendo os campos "severidade" e "equipamento". A tabela utiliza um cabeçalho em azul claro e organiza as informações de forma comparativa para facilitar a compreensão dos elementos essenciais de um prompt estruturado.

Dois recursos adicionais aparecem com frequência. Os exemplos few-shot calibram tanto o formato quanto a fronteira de decisão em casos ambíguos, e costumam render mais que instruções longas em prosa. A decomposição quebra uma tarefa complexa em passos menores e, às vezes, pede que o modelo exponha o raciocínio intermediário antes da resposta. Esse raciocínio exposto melhora o desempenho em tarefas que exigem várias etapas, mas convém uma ressalva que o módulo M5 desenvolve: o texto do raciocínio é uma verbalização plausível, não um registro fiel da computação interna do modelo, e não deve ser tratado como explicação auditável.

A instrução de sistema concentra ainda a política de resposta, e é onde se inscrevem as restrições que o sistema não deve violar: recusar pedidos fora de escopo, não inventar valores ausentes, não extrapolar além dos dados fornecidos. Essas restrições são frágeis quando expressas apenas em linguagem natural, porque o modelo as segue por

regularidade estatística, e não por garantia. Em cenários sensíveis, a política escrita no prompt é a primeira linha de defesa, não a única, e precisa ser reforçada por validação determinista a jusante, fora do modelo, onde uma saída que viole o contrato é rejeitada antes de seguir no pipeline.

Exemplo aplicado

Triagem de chamados de manutenção

Uma planta industrial recebe milhares de chamados de manutenção em texto livre. A tarefa é classificar cada chamado por severidade e extrair o equipamento citado, devolvendo um JSON com os campos severidade (baixa, média, alta) e equipamento. O conhecimento necessário já está no modelo, o volume tolera uma chamada por chamado e o formato é controlável por contrato de saída. É um caso típico em que prompting estruturado resolve sem RAG nem fine-tuning.

Prompting estruturado é o padrão indicado quando três condições se sustentam ao mesmo tempo: o conhecimento necessário já está no pré-treino do modelo, o formato de saída é controlável por contrato, e o volume e a latência toleram uma chamada de modelo por item. Sua maior virtude é o custo de montagem quase nulo; sua principal limitação é a reprodutibilidade, porque a saída pode variar com a redação do prompt e com a versão do modelo, o que exige fixar a versão e manter um conjunto de exemplos de regressão.

Os pontos de ruptura do padrão são tão informativos quanto suas condições de uso, porque cada um deles aponta para outro padrão. Quando o conhecimento necessário não cabe na janela de contexto, ou é privado ou recente demais para estar no pré-treino, ou quando a resposta precisa citar a fonte de onde veio, o prompting sozinho não resolve e o projeto caminha para RAG. Quando o que falta não é conhecimento, mas um comportamento ou formato consistente que nenhum prompt estabiliza, o caminho é o fine-tuning leve. Reconhecer essas rupturas em uma especificação é o objetivo da seção 1.7.

Armadilha comum

A engenharia de prompt sem teto. Ao primeiro resultado insatisfatório, a reação frequente é reescrever o prompt indefinidamente. Quando o problema real é ausência de conhecimento (o modelo não viu aquele manual interno) ou ausência de comportamento estável, nenhuma reescrita resolve, e o custo de iteração cresce sem convergir. A decisão correta não é melhorar o prompt, e sim trocar de padrão; confundir os dois é o erro decisório mais comum desta camada.

Para aprofundar

WENG, L. Prompt Engineering. lilianweng.github.io/posts/2023-03-15-prompt-engineering, 2023. O tratamento aprofundado de regimes de prompting e avaliação de saídas é deferido ao curso de LLMs da Fase 2 do programa.

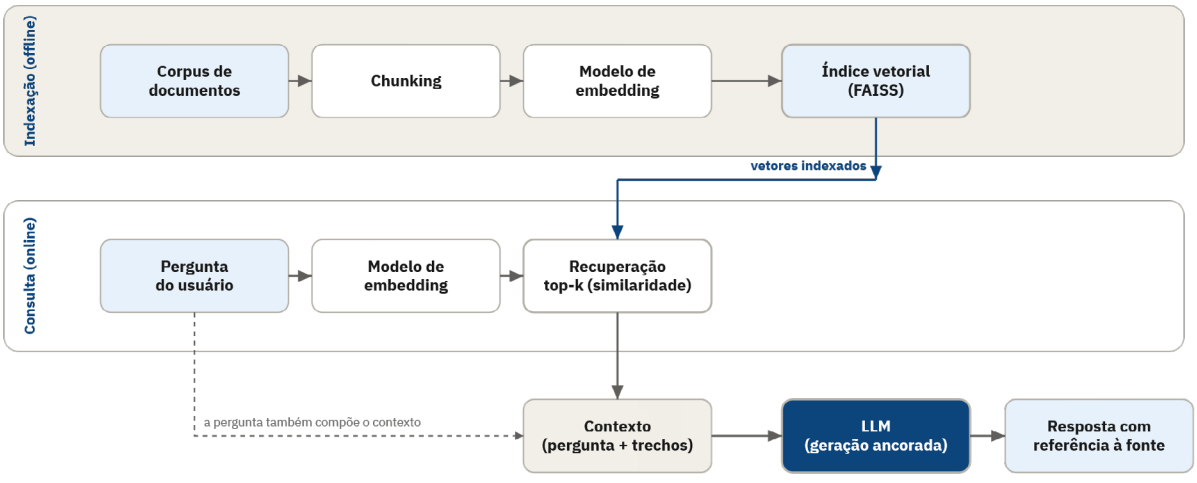
2. RAG Mínimo

Esta seção pertence à camada aplicada e tem por objetivo que você reconheça o RAG como o padrão que injeta conhecimento externo na geração, identifique as etapas do seu pipeline mínimo e os pontos em que ele degrada. O notebook IA-010 instrumenta exatamente este padrão.

RAG, sigla de retrieval-augmented generation, é o padrão que resolve a primeira grande ruptura do prompting: a ausência de conhecimento. Em vez de confiar apenas no que o modelo memorizou no pré-treino, o sistema recupera, a cada pergunta, os trechos mais relevantes de um corpus próprio e os entrega ao modelo como contexto adicional. A geração passa a ser ancorada nesses trechos, o que reduz a alucinação e permite citar a fonte. O conhecimento deixa de estar congelado nos pesos e passa a viver num índice que pode ser atualizado sem retreinar nada.

O pipeline mínimo tem duas raias, como mostra a figura 1 deste ebook. A raia de indexação roda offline, uma vez por versão do corpus: cada documento é dividido em trechos por chunking, cada trecho é convertido em um vetor por um modelo de embedding, e os vetores são guardados em um índice vetorial. A raia de consulta roda a cada pergunta: a pergunta passa pelo mesmo modelo de embedding, o índice devolve os k trechos de maior similaridade, esses trechos são montados junto da pergunta em um contexto, e só então o modelo gera a resposta. O qualificador mínimo indica a ausência de refinamentos como reordenação dos resultados e reescrita da consulta, deixados para o curso de eixo da Fase 2.

Figura 1 - Pipeline de RAG em alto nível.



RAG mínimo: índice in-memory, sem reordenação (reranking) nem reescrita de consulta.

Fonte: Autor.

Para Cego Ver: A imagem apresenta um diagrama intitulado "Figura M1.fig-2. Pipeline de RAG em alto nível", que ilustra o fluxo simplificado de funcionamento de um sistema de Retrieval-Augmented Generation (RAG). O processo é dividido em duas etapas principais. Na parte superior, chamada Indexação (offline), um conjunto de documentos passa pelas etapas de fragmentação em partes menores (chunking), geração de representações vetoriais (embeddings) e armazenamento em um índice vetorial identificado como FAISS, formando uma base de vetores indexados. Na parte inferior, denominada Consulta (online), a pergunta do usuário também é convertida em um vetor por um modelo de embedding e utilizada para recuperar, por similaridade, os trechos mais relevantes do índice vetorial, em uma etapa chamada Recuperação top-k. Esses trechos são combinados com a própria pergunta, formando um contexto que é enviado ao modelo de linguagem (LLM), responsável por gerar uma resposta fundamentada nas informações recuperadas. Como resultado, o sistema produz uma resposta com referência à fonte, indicando que a geração está ancorada nos documentos consultados. Setas conectam todas as etapas, destacando o fluxo contínuo entre indexação, recuperação de informações e geração da resposta. Ao final da figura, há a observação de que o exemplo representa um RAG mínimo, sem reordenação dos resultados (reranking) nem reescrita da consulta. Abaixo do diagrama, aparece a indicação "Fonte: autor."

Sem o passo de recuperação, o modelo responderia apenas com seu conhecimento paramétrico; a recuperação é o que ancora a resposta na fonte.

GLOSSÁRIO

Chunking

Chunking é a divisão de cada documento do corpus em trechos menores antes da indexação, de modo que a recuperação devolva passagens específicas, e não documentos inteiros. O tamanho do trecho e a sobreposição entre trechos vizinhos são decisões de projeto que afetam diretamente a qualidade da recuperação.

A etapa de recuperação é o coração do padrão, e também seu ponto mais frágil. A busca é por similaridade no espaço de embeddings: trechos cujo vetor está próximo do vetor da pergunta são considerados relevantes. Esse mecanismo é poderoso porque captura proximidade de sentido, e não apenas coincidência de palavras, mas é cego ao que não recuperou. Se o trecho que contém a resposta não entra entre os k recuperados, o modelo gera com base em material irrelevante e a resposta fica errada com aparência de segurança. A regra prática é direta: a qualidade da resposta nunca supera a qualidade da recuperação.

Vale uma intuição sobre o que significa similaridade aqui. Cada embedding é um ponto em um espaço de muitas dimensões, e a proximidade entre dois pontos mede proximidade de sentido, não de grafia. Por isso uma pergunta sobre falha de rolamento pode recuperar um trecho que fala em mancal desgastado, ainda que nenhuma palavra coincida. O número k de trechos recuperados é a primeira alavanca de ajuste: um k baixo arrisca deixar de fora a passagem certa, um k alto enche o contexto de ruído e dilui o sinal, além de encarecer a geração. Não há valor universal; calibra-se k contra exemplos reais do domínio.

As decisões de chunking são a segunda alavanca, e talvez a mais subestimada. Trechos longos demais misturam assuntos e degradam a precisão da recuperação; trechos curtos demais fragmentam a informação e perdem o contexto necessário para responder. Uma pequena sobreposição entre trechos vizinhos evita cortar uma frase ou um procedimento ao meio. Como a recuperação é cega ao que não devolveu, avaliar o RAG começa antes da resposta: mede-se se o trecho que contém a informação correta foi

de fato recuperado, porque uma geração impecável sobre um contexto errado continua sendo uma resposta errada.

Exemplo aplicado

Assistente sobre documentação técnica

Uma fabricante de equipamentos mantém milhares de páginas de manuais que mudam a cada revisão de produto. O objetivo é responder perguntas de operadores com base nessa documentação e indicar a página de origem. Treinar o conhecimento nos pesos seria caro e ficaria desatualizado a cada revisão; RAG indexa os manuais, recupera os trechos pertinentes por pergunta e ancora a resposta neles, com rastreabilidade da fonte. Quando o manual muda, basta reindexar.

RAG é o padrão indicado quando o conhecimento necessário é privado, volumoso ou muda mais rápido do que o ciclo de retreino do modelo, e quando a solução precisa apontar a origem da informação. Em setores regulados, essa rastreabilidade deixa de ser conveniência e vira requisito. O custo do padrão é estrutural: há a infraestrutura do índice para construir e manter, a latência adicional de uma busca antes de cada geração, e a sensibilidade do resultado às decisões de chunking e ao número k de trechos recuperados.

É importante delimitar o que RAG não faz. Ele acrescenta conhecimento ao contexto, mas não altera o comportamento do modelo: não muda seu estilo, seu tom nem o formato com que ele escreve. Um sistema que responde no registro errado, ou que precisa de um formato muito específico de saída, não melhora por recuperar mais trechos, porque o problema não é de conhecimento. Esse é o limite que separa RAG do terceiro padrão, o fine-tuning leve, tratado na próxima seção.

A ancoragem é o que dá ao RAG sua propriedade mais valiosa em PD&I: a rastreabilidade. Como cada trecho do contexto vem de um documento identificável, a resposta pode apontar de onde veio cada afirmação, e um revisor pode conferir. Isso muda a natureza do erro tolerável. Um sistema que alucina sem fonte é difícil de auditar; um sistema que cita a passagem permite que o usuário valide a citação, mesmo quando o modelo interpreta mal o trecho recuperado. Em setores regulados, é essa rastreabilidade, e não a redução de custo, que costuma justificar a adoção do padrão.

Armadilha comum

Confundir falta de conhecimento com falta de comportamento. Equipes adotam RAG esperando que ele corrija o estilo ou o formato das respostas, e se frustram quando recuperar mais contexto não muda o registro do texto. RAG resolve o que o modelo não sabe, não o jeito como ele escreve. Tratar os dois problemas como um só leva a investir no padrão errado e a concluir, equivocadamente, que a abordagem generativa não serve para o caso.

Para aprofundar

LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020, arXiv:2005.11401; e a revisão de GAO, Y. et al. arXiv:2312.10997, 2023. O detalhamento de reordenação, reescrita de consulta e avaliação de RAG é deferido ao curso de LLMs da Fase 2.

3. Fine-tuning leve

Esta seção pertence à camada aplicada e tem por objetivo que você reconheça o fine-tuning leve como o padrão que ajusta o comportamento do modelo, distinguindo-o de RAG e situando quando o custo de treinar se justifica diante das alternativas.

Fine-tuning é o ajuste dos pesos de um modelo pré-treinado a partir de exemplos próprios, para que ele passe a se comportar de uma forma específica. O qualificador leve é o que torna o padrão viável em PD&I: em vez de reescrever todos os bilhões de parâmetros, ajusta-se apenas uma pequena fração deles, com técnicas de ajuste eficiente em parâmetros que cabem em hardware modesto e em orçamentos realistas. Diferente dos dois padrões anteriores, aqui o conhecimento muda de lugar: parte do comportamento desejado passa a residir nos pesos, e não no prompt nem num índice externo.

GLOSSÁRIO

LoRA

LoRA (low-rank adaptation) é a técnica de ajuste eficiente em parâmetros mais usada: em vez de alterar os pesos originais do modelo, treina-se um pequeno conjunto de matrizes adicionais de baixo posto, que são somadas ao modelo na inferência. O modelo-base permanece intacto, e o adaptador resultante ocupa uma fração mínima do espaço.

A família dessas técnicas é chamada de PEFT, ajuste fino eficiente em parâmetros, e o regime de treino mais comum em projetos aplicados é o ajuste supervisionado, no qual o modelo aprende com pares de entrada e saída exemplares preparados pela equipe. Regimes mais sofisticados de alinhamento por preferência humana existem e são relevantes em produtos de larga escala, mas sua implementação é deferida ao curso de eixo da Fase 2; no curso base, o que importa é reconhecer o fine-tuning leve como uma opção de projeto, com seu custo e seus pré-requisitos próprios.

O fine-tuning leve é indicado quando o que falta não é conhecimento, mas comportamento estável. Três sinais o recomendam. Primeiro, quando há um estilo, tom ou formato de saída muito específico que nenhum prompt consegue fixar de forma consistente. Segundo, quando uma tarefa especializada se repete em alto volume e vale internalizar as instruções nos pesos, encurtando o prompt e reduzindo o custo por chamada. Terceiro, quando a latência ou o custo de prompts longos, repetidos milhões de vezes, supera o custo de treinar uma vez. Note que os três sinais são sobre como o modelo responde, não sobre o que ele sabe.

Convém situar o leve diante da alternativa pesada. O fine-tuning completo reescreve todos os pesos do modelo e exige infraestrutura, dados e orçamento que raramente se justificam em PD&I aplicada, além de produzir um modelo inteiro novo para servir e versionar. As técnicas eficientes em parâmetros invertem essa economia: o modelo-base permanece único e compartilhado, e cada tarefa especializada vira um adaptador pequeno que se acopla na inferência. Isso permite manter vários comportamentos especializados sobre o mesmo modelo, trocando apenas o adaptador, com custo de armazenamento e de troca próximos de zero.

Quadro 2 - Os três padrões aplicados lado a lado.

Eixo	Prompting estruturado	RAG mínimo	Fine-tuning leve
O que altera	A entrada do modelo	O contexto recuperado	Os pesos do modelo
Resolve falta de	Nada externo; usa o que o modelo já sabe	Conhecimento	Comportamento ou formato
Dados necessários	Poucos exemplos no prompt	Corpus indexável	Pares rotulados de qualidade
Atualização	Reescrever o prompt	Reindexar o corpus	Retreinar o adaptador
Rastreabilidade da fonte	Não	Sim, pela recuperação	Não
Custo de montagem	Quase nulo	Médio (índice e latência)	Alto (dados e treino)

Para Cego Ver: A imagem apresenta uma tabela intitulada "Quadro 2. Os três padrões aplicados lado a lado", que compara três abordagens utilizadas em aplicações com modelos de linguagem: Prompting estruturado, RAG mínimo e Fine-tuning leve. A comparação é organizada em linhas que representam diferentes aspectos. No eixo "O que altera", o prompting modifica a entrada do modelo, o RAG modifica o contexto recuperado e o fine-tuning altera os pesos do modelo. Em "Resolve falta de", o prompting aproveita apenas o conhecimento que o modelo já possui, o RAG supre a ausência de conhecimento por meio de documentos recuperados e o fine-tuning ajusta o comportamento ou o formato das respostas. Na linha "Dados necessários", o prompting requer poucos exemplos inseridos no prompt, o RAG depende de um corpus de documentos indexável e o fine-tuning exige pares de exemplos rotulados de qualidade. Em "Atualização", o prompting demanda apenas a reescrita do prompt, o RAG requer a reindexação do corpus e o fine-tuning necessita de novo treinamento do adaptador. Quanto à "Rastreabilidade da fonte", apenas o RAG permite identificar a origem das informações recuperadas. Por fim, na comparação de "Custo de montagem", o prompting apresenta custo quase nulo, o RAG possui custo intermediário devido à criação do índice e à latência da recuperação, enquanto o fine-tuning apresenta custo elevado por exigir preparação de dados e treinamento. A tabela utiliza cabeçalho em azul claro e organiza as informações em colunas para facilitar a comparação entre as três abordagens.

Exemplo aplicado

Laudos em formato fixo

Um laboratório precisa que cada laudo siga uma estrutura rígida de seções, terminologia e ordem, com tom uniforme entre analistas. Tentativas de fixar esse formato por prompt produziram variação inaceitável entre execuções. Um fine-tuning leve sobre algumas centenas de laudos bem redigidos internaliza o formato nos pesos, estabiliza a saída e ainda encurta o prompt de cada geração. O conhecimento clínico vem dos dados de entrada; o que se ensinou ao modelo foi a forma de escrever.

A aritmética de custo costuma ser o argumento decisivo, e vale torná-la explícita. Prompting e RAG não têm custo de treino, mas pagam, a cada chamada, pelos tokens de instruções e de contexto; em altíssimo volume, esse custo recorrente domina. O fine-tuning leve concentra o gasto em um treino único e, ao internalizar instruções nos pesos, encurta o prompt de toda chamada subsequente, reduzindo o custo marginal. A decisão entre pagar uma vez para treinar ou pagar sempre por prompts mais longos depende do volume esperado e do horizonte do projeto, e é o tipo de compromisso que a camada decisória registra.

Avaliar um modelo ajustado pede o mesmo rigor de qualquer modelo supervisionado, com um cuidado extra. Mede-se o ganho na tarefa-alvo, mas verifica-se também se o ajuste não degradou capacidades gerais que o projeto ainda usa, fenômeno conhecido como esquecimento catastrófico. Um conjunto de avaliação que cubra tanto a tarefa especializada quanto casos gerais de controle é a salvaguarda mínima; sem ele, o modelo pode parecer melhor no que foi treinado e pior em tudo o mais, e a regressão só aparece em produção.

Os pontos de ruptura do fine-tuning leve são mais severos do que os dos outros padrões, e por isso ele raramente é o ponto de partida. Ele exige dados rotulados de qualidade, cuja curadoria costuma ser o maior custo escondido do projeto; o módulo M3 trata essa questão em profundidade. Não incorpora bem conhecimento que muda, porque cada atualização pede novo treino, ao contrário da reindexação barata do RAG. Carrega risco de overfitting e de esquecer capacidades gerais do modelo se o treino for desequilibrado. E não oferece rastreabilidade da fonte, já que o que aprendeu se dissolveu nos pesos.

> **Armadilha comum**
>
> Fazer fine-tuning para ensinar fatos. A tentação de treinar o modelo com documentos internos para que ele os memorize confunde os papéis dos padrões. Conhecimento que muda pertence a um índice de RAG, que se reindexa em minutos; gravá-lo nos pesos resulta em respostas desatualizadas, caras de corrigir e sem citação de fonte. Fine-tuning ensina comportamento, RAG fornece conhecimento; inverter os dois é a confusão decisória central desta seção.

4. Reconhecimento de padrões em uma especificação

Este tópico aborda a camada aplicada e fecha o reconhecimento dos três padrões: o objetivo é que você leia uma especificação de solução e identifique qual padrão está em jogo, sabendo que eles se compõem em vez de se excluírem. É a ponte para a camada decisória do módulo.

Os tópicos anteriores apresentaram os padrões um a um; o trabalho aplicado real é o inverso: diante de uma especificação, decidir qual deles a situação pede. A chave para essa leitura é separar duas perguntas que costumam vir embaralhadas. A primeira é sobre conhecimento: o que o sistema precisa saber está no modelo, ou precisa vir de fora? A segunda é sobre comportamento: o jeito como o sistema deve responder, em formato, estilo e consistência, é alcançável por instrução, ou precisa ser aprendido? Conhecimento e comportamento são eixos independentes, e cada padrão atua sobre um deles.

A heurística de leitura segue uma ordem de custo crescente e essa ordem não é arbitrária. Comece sempre por prompting estruturado, porque é o padrão de menor custo e muitas vezes suficiente. Suba para RAG quando a especificação revelar necessidade de conhecimento privado, volumoso, recente ou rastreável que o prompt não consegue carregar. Considere fine-tuning leve quando o que falta for comportamento estável que nem prompting nem RAG entregam. Essa progressão ecoa, na lente criativa, a mesma disciplina que o módulo M2 aplica à lente da autonomia: não suba de patamar de custo antes que o patamar atual se mostre insuficiente.

Na prática, a leitura de uma especificação se beneficia de uma sequência curta de perguntas diagnósticas. Primeiro: o conhecimento de que a solução precisa é geral e estável, ou é privado, volumoso ou mutável? Segundo: a saída precisa ser verificável por uma máquina a jusante, ou citar uma fonte? Terceiro: o estilo, o tom e o formato são alcançáveis por instrução, ou já se mostraram instáveis sob prompting? Quarto: qual o volume esperado de chamadas e o horizonte do projeto? As respostas mapeiam, quase diretamente, os sinais do quadro seguinte para os padrões correspondentes.

4.1 Sinais na especificação e o padrão que cada um indica.

Sinal no texto da especificação	Padrão indicado
O conhecimento necessário é geral e já consolidado	Prompting estruturado
A saída precisa de formato verificável para um pipeline	Prompting estruturado (contrato de saída)
Há base de conhecimento privada, extensa ou mutável	RAG mínimo
A resposta precisa citar a fonte de origem	RAG mínimo
O estilo ou formato precisa ser uniforme e nenhum prompt fixa	Fine-tuning leve
Tarefa especializada repetida em altíssimo volume	Fine-tuning leve (custo por chamada)

Para Cego Ver: A imagem apresenta uma tabela intitulada "Quadro 3. Sinais na especificação e o padrão que cada um indica", organizada em duas colunas: "Sinal no texto da especificação" e "Padrão indicado". A tabela relaciona diferentes características identificadas em uma especificação com a abordagem mais adequada para aplicações baseadas em modelos de linguagem. Os exemplos apresentados indicam que, quando o conhecimento necessário é geral e já consolidado, recomenda-se o Prompting estruturado. Quando a saída precisa seguir um formato verificável para integração com um pipeline, a indicação é Prompting estruturado com contrato de saída. Se houver uma base de conhecimento privada, extensa ou frequentemente atualizada, a abordagem recomendada é o RAG mínimo. Quando a resposta precisa citar a fonte de origem das informações, também se recomenda o RAG mínimo. Já nos casos em que o estilo ou o formato das respostas deve permanecer uniforme, sem depender de um prompt fixo, o padrão indicado é o Fine-tuning leve. Por fim, para tarefas altamente especializadas executadas repetidamente em grande volume, a recomendação é utilizar Fine-tuning leve, destacando seu potencial para reduzir o custo por chamada. A tabela utiliza um cabeçalho em azul claro e organiza as informações de forma comparativa, facilitando a identificação da abordagem mais adequada para diferentes cenários de uso.

O passo conceitualmente mais importante desta seção é abandonar a ideia de que os padrões são alternativas mutuamente exclusivas. Eles se compõem. O prompting estruturado está presente em todas as soluções, porque mesmo um sistema de RAG ou um modelo ajustado ainda recebe instruções e contrato de saída. RAG e fine-tuning combinam-se com naturalidade: um assistente pode ter o comportamento ajustado por fine-tuning leve para escrever no registro certo e, ao mesmo tempo, recuperar conhecimento atualizado por RAG. Ler uma especificação é menos escolher uma caixa e mais mapear quais camadas a solução vai empilhar.

Exemplo aplicado

Assistente regulado de uma seguradora

Uma seguradora quer um assistente que responda dúvidas de clientes sobre apólices. As respostas precisam citar a cláusula exata, o que aponta RAG sobre o acervo de apólices. Precisam também seguir um tom padronizado e um formato fixo de resposta exigido pela área de compliance, o que nenhuma instrução estabilizou, sinal de fine-tuning leve. E toda chamada parte de um prompt estruturado com contrato de saída. A especificação pede os três padrões compostos, não um só.

Reconhecer o padrão correto na especificação é também antecipar o custo que ele impõe ao projeto, e essa é a transição para a camada decisória. Cada padrão carrega um perfil distinto de custo, latência, dados e rastreabilidade, e a escolha entre eles, ou a ordem em que se empilham, precisa ser registrada e defendida diante de quem patrocina o projeto. A camada decisória do módulo, tratada no texto-base do próximo e-book e instrumentada pelos templates de decisão, organiza essa defesa; o notebook Colab a seguir, ao comparar uma resposta gerada com e sem contexto recuperado, dá a evidência empírica de quando o padrão de RAG muda o resultado e quando não muda.

A ordem em que os padrões se empilham também importa, e não é livre. O prompting estruturado é a camada externa de qualquer solução, porque mesmo um modelo ajustado e municiado por RAG ainda recebe instrução e contrato de saída. RAG e fine-tuning operam em planos distintos e não concorrem: um decide o que o modelo vê de conhecimento, o outro decide como o modelo se comporta. Quando ambos são necessários, a sequência natural é ajustar o comportamento uma vez e recuperar o conhecimento a cada chamada, mantendo o índice como a peça que se atualiza com frequência e o adaptador como a peça estável.

Por fim, reconhecer o padrão é também o primeiro passo da decisão maior que abre a camada decisória do módulo, entre uma solução generativa e um modelo de aprendiza-

do de máquina clássico. Nem todo problema que admite geração pede um modelo generativo: uma classificação de alto volume com rótulos disponíveis costuma ser melhor servida por um classificador clássico, mais barato, mais rápido e mais fácil de explicar. Saber qual padrão generativo a especificação pediria é o que permite comparar, com honestidade, o custo dessa rota contra o da alternativa clássica, comparação que a competência M1.C3 desenvolve.

Armadilha comum

O reflexo do mais novo. Diante de um problema, equipes tendem a alcançar primeiro o padrão mais sofisticado que conhecem, montando RAG ou fine-tuning sem testar se um prompt estruturado já resolveria. A consequência é custo e complexidade injustificados, além de mais superfícies de falha para manter. A decisão defensável parte sempre do padrão mais simples e só sobe quando há evidência de que ele não basta; o ônus da prova é de quem quer subir, não de quem fica.

Para aprofundar

a articulação destas decisões com cronograma, custo e validação de um projeto é o objeto da camada decisória do módulo (texto-base IA-011 e templates T1.1 a T1.3). A leitura comparada dos cinco eixos decisórios fecha a competência M1.C3.

Conclusão

As quatro seções deste texto-base percorreram, na camada aplicada da lente criativa, os três padrões que estruturam soluções generativas em PD&I industrial e a leitura que os reconhece numa especificação. Prompting estruturado compõe a entrada do modelo e resolve o que ele já sabe; RAG injeta conhecimento externo e ancora a resposta numa fonte rastreável; fine-tuning leve ajusta o comportamento gravando parte dele nos pesos. Os três não competem: compõem-se, e a disciplina aplicada é começar pelo mais barato e subir apenas com evidência.

O ganho prático deste reconhecimento aparece no encontro com o squad. Saber nomear o padrão certo, e seu ponto de ruptura, evita o desperdício de montar uma arquitetura cara para um problema que um contrato de saída resolveria, e evita o oposto, insistir num prompt onde faltava conhecimento ou comportamento. Essa leitura é o que a camada decisória do módulo converte em escolha defensável, com eixos explícitos de custo, latência, reprodutibilidade, precisão e disponibilidade de dados.

A próxima peça da camada aplicada é o notebook Colab a seguir, que instrumenta o

padrão de RAG em escala mínima: você indexará um corpus público, recuperará trechos por similaridade e comparará a resposta do modelo com e sem o contexto recuperado, observando na prática quando a recuperação muda o resultado. Em seguida, a camada decisória do módulo organiza as cinco decisões correlatas em torno da escolha GenAI versus ML clássico.

REFERÊNCIAS

Bibliografia mínima

VASWANI, A. et al. Attention Is All You Need. NeurIPS, 2017. Preprint em https://arXiv.org/abs/1706.03762.

LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. Preprint em https://arXiv.org/abs/2005.11401.

OUYANG, L. et al. Training Language Models to Follow Instructions with Human Feedback (InstructGPT). NeurIPS, 2022. Preprint em https://arXiv.org/abs/2203.02155.

GAO, Y. et al. Retrieval-Augmented Generation for Large Language Models: A Survey. Preprint em https://arxiv.org/abs/2312.10997, 2023.

Bibliografia complementar

BOMMASANI, R. et al. On the Opportunities and Risks of Foundation Models. CRFM Stanford, 2021. Preprint em https://arXiv.org/abs/2108.07258.

HO, J.; JAIN, A.; ABBEEL, P. Denoising Diffusion Probabilistic Models. NeurIPS, 2020. Preprint em https://arXiv.org/abs/2006.11239.

RAFAILOV, R. et al. Direct Preference Optimization. NeurIPS, 2023. Preprint em https://arXiv.org/abs/2305.18290.

UNIÃO EUROPEIA. Regulamento (UE) 2024/1689 (EU AI Act). Artigos sobre GPAI (51 a 55). Texto integral em https://eur-lex.europa.eu.

BRASIL. Senado Federal. Projeto de Lei n. 2338/2023. Marco regulatório da IA no Brasil. Texto integral em portais públicos do Senado.

Industr.IA

Residência em TIC 49



Executor iRede

Executor Atlântico

Coordenador Softex

Financiador MCTI Futuro

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil — Do lado do povo brasileiro

Projeto apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991.